=== UNIT 2: CONTROL STRUCTURES, BRANCHING, LOOPS & FORMATTED I/O ===

>> 2.1 Conditional Branching Statements

Control structures govern the execution flow of programs based on dynamic computational conditions.
* The if-else Statement: Tests a relational/logical expression. If non-zero (true), if-block executes; otherwise, else-block executes. Dangling-Else Ambiguity: In nested if statements without braces, compiler pairs an else clause with closest preceding unmatched if.
* The switch-case Construct: Multi-way branching based on integral expression (int or char only; floats and strings invalid). Matches against constant integral case labels. break statement is mandatory to prevent fall-through. Optional default label executes when no case matches.

>> 2.2 Iteration and Looping Constructs

* Entry-Controlled Loops:
- while Loop: Evaluates condition BEFORE executing loop body: while (condition) { body; }. If initial condition is false, body executes zero times.
- for Loop: Compact loop syntax: for (init; test; update) { body; }. Any or all three expressions may be omitted; for(;;) creates infinite loop.
* Exit-Controlled Loop:
- do-while Loop: Evaluates condition AFTER executing body: do { body; } while (condition);. Guaranteed to execute at least once. Mandatory semicolon follows closing parenthesis.
* Jump Statements:
- break: Terminates innermost loop or switch construct immediately.
- continue: Skips remaining statements in current iteration and jumps directly to loop condition test (or step expression in for loops).
- goto: Unconditional jump to labeled statement within same function scope. Strongly discouraged in structured programming.

>> 2.3 Formatted and Unformatted I/O Functions

* printf(): Formatted stream output:
printf("%[flags][width][.precision][length]specifier", args). Width specifies minimum characters; precision specifies decimal digits for floats or string truncation length.
* scanf(): Formatted stream input: scanf("%specifier", &variable). Requires address-of operator (&) for primitive types, except for array/string names. Buffer issues: Trailing newline ('\n') left by scanf() can inadvertently skip subsequent

getchar() or gets() calls.
* Unformatted I/O: getchar(), putchar(), gets() (deprecated due to buffer overflow vulnerability), fgets(), puts().

[PROGRAM 1: Prime Number Verification and Sieve of Eratosthenes]

Problem: Write an optimized C program to: (1) Verify if a given integer N is prime in O(sqrt(N)) time, (2) Generate all prime numbers up to limit M using Sieve of Eratosthenes.
Source Code:

```
#include <stdio.h>
#include <stdbool.h>
#include <string.h>
bool isPrime(int n) {
if (n <= 1) return false;
if (n <= 3) return true;
if (n % 2 == 0 || n % 3 == 0) return false;
for (int i = 5; i * i <= n; i += 6) {
if (n % i == 0 || n % (i + 2) == 0)
return false;
}
return true;
}
void sieveOfEratosthenes(int limit) {
bool prime[limit + 1];
memset(prime, true, sizeof(prime));
for (int p = 2; p * p <= limit; p++) {
if (prime[p] == true) {
for (int i = p * p; i <= limit; i += p)
prime[i] = false;
}
}
printf("Primes up to %d: ", limit);
for (int p = 2; p <= limit; p++)
if (prime[p]) printf("%d ", p);
printf("\n");
}
int main() {
int testVal = 29;
printf("%d is %s\n", testVal, isPrime(testVal) ? "PRIME" : "COMPOSITE");
sieveOfEratosthenes(30);
return 0;
}
```

Output:

29 is PRIME
Primes up to 30: 2 3 5 7 11 13 17 19 23 29

[PROGRAM 2: Fibonacci Sequence Generation via Iteration]

Problem: Write a program to generate first N terms of Fibonacci sequence using iterative looping without recursion, and calculate Golden Ratio approximation.
Source Code:

```
#include <stdio.h>
int main() {
int n = 10;
long long first = 0, second = 1, next;
printf("First %d Fibonacci terms:\n", n);
for (int i = 1; i <= n; i++) {
if (i == 1) {
printf("%lld ", first);
continue;
}
if (i == 2) {
printf("%lld ", second);
continue;
}
next = first + second;
first = second;
second = next;
printf("%lld ", next);
}
printf("\nGolden Ratio Approximation (F_%d / F_%d): %.6f\n", n, n-1, (double)second /
first);
return 0;
}
```

Output: First 10 Fibonacci terms: 0 1 1 2 3 5 8 13 21 34 | Golden Ratio Approximation: 1.619048.

[PROGRAM 3: Multi-Way Arithmetic Calculator Using switch-case]

Problem: Implement a menu-driven arithmetic calculator processing binary operands using switch-case, including zero-division error handling.
Source Code:

```
#include <stdio.h>
int main() {
char op;
```

```
double num1, num2, result;
printf("Enter operator (+, -, *, /): ");
scanf(" %c", &op); /* Leading space consumes leftover newline */
printf("Enter two operands: ");
scanf("%lf %lf", &num1, &num2);
switch (op) {
case '+': result = num1 + num2; printf("%.2f + %.2f = %.2f\n", num1, num2, result);
break;
case '-': result = num1 - num2; printf("%.2f - %.2f = %.2f\n", num1, num2, result);
break;
case '*': result = num1 * num2; printf("%.2f * %.2f = %.2f\n", num1, num2, result);
break;
case '/':
if (num2 == 0.0) {
printf("Error: Division by zero is mathematically undefined!\n");
} else {
result = num1 / num2;
printf("%.2f / %.2f = %.2f\n", num1, num2, result);
}
break;
default: printf("Error: Unrecognized operator '%c'\n", op); break;
}
return 0;
}
```

[PROGRAM 4: Pattern Printing Using Nested Loops (Pascal's Triangle)]

Problem: Write a program to print Pascal's Triangle up to N rows using nested for loops and combination formula C(n, k) = C(n, k-1) * (n - k + 1) / k.

Source Code:

```
#include <stdio.h>
void printPascalsTriangle(int rows) {
for (int i = 0; i < rows; i++) {
for (int space = 0; space < rows - i - 1; space++)
printf(" ");
int val = 1;
for (int j = 0; j <= i; j++) {
printf("%4d", val);
val = val * (i - j) / (j + 1);
}
printf("\n");
}
}
int main() {
```

```
printPascalsTriangle(5);
return 0;
}
```

Output Pattern:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

>> 2.4 University Review & Examination Questions

* Q1: Explain the Dangling-Else problem in C programming and how it is resolved syntactically.

Answer: The dangling-else problem arises in nested selection statements when an 'else' keyword follows an inner 'if' inside an outer 'if' without enclosing braces: 'if (cond1) if (cond2) stmt1; else stmt2;'. Semantically, it is ambiguous whether 'else' pairs with outer or inner 'if'. In C, grammar dictates that an 'else' is always associated with the NEAREST PRECEDING UNMATCHED 'if' within the same block scope. Therefore, stmt2 belongs to 'if (cond2)'. To pair else with the outer if, compound braces must be used to close inner if explicitly: 'if (cond1) { if (cond2) stmt1; } else stmt2;'.

* Q2: Compare and contrast entry-controlled loops (for, while) with exit-controlled loops (do-while).

Answer: (1) Execution order: Entry-controlled loops evaluate test conditions BEFORE executing loop body. If condition evaluates to false initially, body is completely bypassed (minimum execution count = 0). Exit-controlled loops (do-while) execute body FIRST before checking test condition at bottom, guaranteeing body executes AT LEAST ONCE. (2) Semicolon syntax: for and while headers do not end with semicolon. do-while requires mandatory terminating semicolon following condition 'do { ... } while (cond);'. (3) Use case: Entry-controlled loops preferred when bounds are known beforehand; do-while ideal for interactive menu-driven programs where options must be displayed before user choice can be tested.

>> 2.5 Key Takeaways and Revision Points

* switch-case requires integral types and break statements to prevent automatic fall-through.
* do-while loops guarantee at least one execution; for loops combine init, test, and update.
* Nested loops handle multi-dimensional tables and geometric patterns.
* scanf requires variable addresses (&); leading whitespace in format specifier consumes buffer newlines.